

1. AI-Powered Drone for Emergency Medicine Delivery

Problem Statement

A government deploys AI-powered drones to deliver emergency medicines in flood-affected regions. The drone must find the safest and fastest route despite blocked roads, changing weather, and varying travel costs.

i. Greedy Best-First Search (GBFS)

Working:

- Selects the path that appears closest to the destination using only the heuristic value $h(n)$ (straight-line distance).
- Ignores the actual travel cost.

Strengths

- Very fast decision-making.
- Uses less memory.
- Suitable for urgent situations requiring immediate action.

Weaknesses

- Does not guarantee the shortest or safest path.
- May choose blocked or expensive routes.
- Performance decreases when heuristic information is inaccurate.

ii. A* Search Algorithm

Working

- Uses both actual path cost and heuristic.
- Evaluation function:

$$f(n) = g(n) + h(n)$$

Where:

- $g(n)$ = Actual cost from start to current node.
- $h(n)$ = Estimated cost to destination.

Advantages

- Finds the optimal route when heuristic is admissible.
- Considers weather, terrain, and blocked roads.

- Produces safer and more reliable routes.

Disadvantages

- More computationally expensive.
- Requires additional memory.

iii. Impact of Heuristic Accuracy

Heuristic Accuracy	Greedy Best-First Search	A* Search
High Accuracy	Very fast and efficient	Fast and optimal
Moderate Accuracy	May choose poor routes	Usually still optimal
Low Accuracy	Easily trapped in bad paths	Performance decreases but remains reliable

iv. Effect of Dynamic Changes

When roads become blocked:

Greedy Best-First Search

- May continue toward blocked routes.
- Often needs complete replanning.

A*

- Updates path cost.
- Recalculates a better route.
- Handles dynamic environments more effectively.

v. Recommended Algorithm

*Recommended: A Search**

Justification

- Produces optimal routes.
- Balances distance and travel cost.
- Adapts better to changing environments.
- Improves safety for emergency medicine delivery.

CODE:

```
from queue import PriorityQueue
```

```
graph = {  
    'A': [('B', 2), ('C', 4)],  
    'B': [('D', 3), ('E', 5)],  
    'C': [('F', 2)],  
    'D': [('G', 4)],  
    'E': [('G', 2)],  
    'F': [('G', 3)],  
    'G': []  
}
```

```
heuristic = {  
    'A': 7,  
    'B': 6,  
    'C': 5,  
    'D': 4,  
    'E': 2,  
    'F': 3,  
    'G': 0  
}
```

```
def astar(start, goal):  
    pq = PriorityQueue()  
    pq.put((0, start))  
  
    cost = {start: 0}  
    parent = {start: None}
```

```
    while not pq.empty():
```

```

_, current = pq.get()

if current == goal:
    break

for neighbor, weight in graph[current]:
    new_cost = cost[current] + weight

    if neighbor not in cost or new_cost < cost[neighbor]:
        cost[neighbor] = new_cost
        priority = new_cost + heuristic[neighbor]
        pq.put((priority, neighbor))
        parent[neighbor] = current

path = []
node = goal

while node:
    path.append(node)
    node = parent[node]

path.reverse()
print("Optimal Path:", path)
print("Total Cost:", cost[goal])
astar('A', 'G')

```

OUTPUT:\

```

===== RESTART: C:/Users/DELL/OneDrive/Attachments/Documents/AI LAB/N
Optimal Path: ['A', 'B', 'D', 'G']
Total Cost: 9
. |

```

2. Smart City Traffic Signal Optimization

Problem Statement

The AI system must optimize traffic signals to minimize:

- Waiting time
- Fuel consumption
- Traffic congestion

Large search space makes traditional search inefficient.

Why Traditional Search is Inefficient

- Millions of signal combinations.
- Traffic changes continuously.
- Exhaustive search requires excessive computation.
- Real-time response is impossible.

i. Hill Climbing

Working

- Starts with an initial signal timing.
- Makes small adjustments.
- Accepts changes only if they improve traffic flow.

Advantages

- Simple.
- Fast.
- Requires little memory.

Limitations

- May stop at local optimum.
- Cannot escape plateaus.
- Sensitive to starting solution.

ii. Local Maxima, Plateaus and Ridges

Local Maximum

A solution better than nearby solutions but not globally best.

Example

Traffic improves in one area while worsening citywide.

Plateau

Flat search space where many neighboring solutions have identical performance.

Example

Changing green light duration has no noticeable effect.

Ridge

Improvement requires moving in multiple directions.

Example

Changing only one intersection gives little improvement, while changing several together greatly reduces congestion.

iii. Simulated Annealing and Genetic Algorithms

Simulated Annealing

- Sometimes accepts worse solutions.
- Escapes local maxima.
- Probability of accepting worse solutions decreases over time.

Genetic Algorithm

Uses:

- Selection
- Crossover
- Mutation

Advantages:

- Searches many solutions simultaneously.
- Avoids local optimum.

- Excellent for large optimization problems.

iv. Recommended Approach

Recommended: Genetic Algorithm

Reason

- Highly scalable.
- Works well with very large search spaces.
- Handles dynamic traffic conditions.
- Produces near-optimal solutions efficiently.

CODE:

```
import random
```

```
def objective(x):
```

```
    return -(x - 30) ** 2 + 100
```

```
current = random.randint(0, 60)
```

```
while True:
```

```
    left = current - 1
```

```
    right = current + 1
```

```
    best = current
```

```
    if objective(left) > objective(best):
```

```
        best = left
```

```
    if objective(right) > objective(best):
```

```
        best = right
```

```
if best == current:
    break

current = best

print("Optimal Signal Timing:", current)
print("Fitness:", objective(current))
```

OUTPUT:

```
===== RESTART: C:/WASTE/NM.py =====
Optimal Signal Timing: 30
Fitness: 100
.
```


3. Autonomous Mars Rover

Problem Statement

The rover must:

- Explore unknown terrain.
- Avoid hazards.
- Collect samples.
- Operate without a complete map.

i. Why Online Search?

Offline search assumes:

- Complete map available.
- Static environment.

Mars rover has:

- Unknown terrain.
- Limited sensing.
- Continuous discoveries.

Therefore, online search is required.

ii. Challenges

Partial Observability

- Rover only sees nearby terrain.
- Hidden obstacles remain unknown.

Dynamic Environment

- Dust storms.
- Loose rocks.
- Changing terrain conditions.

iii. Internal Model

The rover:

- Stores explored locations.
- Marks hazards.
- Updates safe paths.
- Builds a map while moving.

The internal model improves continuously during exploration.

iv. Comparison

Feature	Uninformed Search	Informed Search	Online Search
Complete Map Needed	Yes	Yes	No
Uses Heuristic	No	Yes	Sometimes
Handles Unknown Areas	No	Limited	Yes
Real-Time Decisions	Poor	Moderate	Excellent

v. Recommended Approach

Recommended: Online Search Agent

Justification

- Handles uncertainty.
- Learns while exploring.
- Adapts to new obstacles.
- Provides safe navigation.

CODE:

```
grid = [
    ['S', '.', '.'],
    ['#', '.', '.'],
    ['.', '.', 'G']
]
```

```

rows = len(grid)
cols = len(grid[0])

visited = set()

def online_search(x, y):
    if x < 0 or y < 0 or x >= rows or y >= cols:
        return False

    if grid[x][y] == '#':
        return False

    if (x, y) in visited:
        return False

    visited.add((x, y))

    print("Visiting:", x, y)

    if grid[x][y] == 'G':
        print("Goal Reached")
        return True

    return (
        online_search(x + 1, y) or
        online_search(x, y + 1) or
        online_search(x - 1, y) or
        online_search(x, y - 1)
    )

```

online_search(0, 0)

OUTPUT:

```
===== RESTART: C:/WASTE/CK.py =====  
Visiting: 0 0  
Visiting: 0 1  
Visiting: 1 1  
Visiting: 2 1  
Visiting: 2 2  
Goal Reached
```

4. University Exam Timetable using CSP

Problem Statement

Create an exam schedule satisfying:

- No overlapping exams.
- Limited halls.
- Faculty availability.
- Subject ordering.
- Student accommodations.

i. CSP Formulation

Variables

Each course examination.

Example:

- CS101
- CS102
- MA101

Domains

Possible:

- Time slots
- Exam halls

Example:

- Morning
- Afternoon
- Evening

Constraints

- No student has overlapping exams.
- Hall capacity must not exceed limit.

- Faculty must be available.
- Some exams occur before others.
- Special students receive suitable arrangements.

ii. Backtracking Search

Steps:

1. Assign first exam.
2. Check constraints.
3. If conflict occurs:
 - Undo assignment.
 - Try another slot.
4. Continue until all exams are scheduled.

iii. Constraint Propagation (Forward Checking)

After assigning an exam:

- Remove conflicting time slots from remaining exams.
- Detect failures early.
- Reduce unnecessary search.

Benefits:

- Faster scheduling.
- Less backtracking.
- Better scalability.

iv. Best Strategy

Backtracking + Forward Checking + Constraint Propagation

Reasons:

- Efficient.
- Reduces search space.
- Handles multiple constraints.

- Suitable for large universities.

CODE:

```
subjects = ["Math", "AI", "DBMS"]
```

```
slots = [1, 2, 3]
```

```
assignment = {}
```

```
def safe(subject, slot):  
    return slot not in assignment.values()
```

```
def solve(index):  
    if index == len(subjects):  
        return True
```

```
    subject = subjects[index]
```

```
    for slot in slots:  
        if safe(subject, slot):  
            assignment[subject] = slot
```

```
        if solve(index + 1):  
            return True
```

```
        del assignment[subject]
```

```
    return False
```

```
solve(0)
```

```
print("Exam Timetable")  
  
for s in assignment:  
    print(s, "-> Slot", assignment[s])
```

OUTPUT:

```
===== RESTART: C:/WASTE/SP.py =====  
Exam Timetable  
Math -> Slot 1  
AI -> Slot 2  
DBMS -> Slot 3
```


5. Strategic Game AI (Minimax & Alpha-Beta Pruning)

Problem Statement

Develop an AI opponent that:

- Plays against humans.
- Makes decisions quickly.
- Handles very large decision trees.

i. Minimax Algorithm

Models the game as a tree.

- MAX player → AI
- MIN player → Opponent

AI chooses the move that maximizes its minimum guaranteed score.

ii. Computational Challenges

- Huge branching factor.
- Deep search tree.
- High memory usage.
- Slow response without optimization.

Time complexity:

$$O(b^d)$$

where:

- **b** = Branching factor
- **d** = Search depth

iii. Alpha-Beta Pruning

Maintains:

- **Alpha (α)** = Best score for MAX.
- **Beta (β)** = Best score for MIN.

If:

$$\alpha \geq \beta$$

Remaining branches are ignored because they cannot improve the final decision.

Benefits

- Reduces unnecessary node evaluation.
- Searches deeper within the same time.
- Produces the same optimal move as Minimax.

iv. Evaluation Function

Example:

$$\begin{aligned} \text{Score} = & (5 \times \text{Resources}) + (10 \times \text{ArmyStrength}) + (8 \times \text{TerritoryControl}) \\ & - (6 \times \text{EnemyStrength}) \end{aligned}$$

Factors considered:

- Resources.
- Army strength.
- Territory control.
- Enemy strength.
- Remaining health.
- Strategic position.

v. Recommended Strategy

Recommended: Minimax with Alpha-Beta Pruning

Justification

- Produces optimal decisions.
- Greatly reduces computation.
- Meets real-time constraints.
- Widely used in modern game AI.
- Balances decision quality and execution speed effectively.

CODE:

```
import math
```

```
tree = [3, 5, 6, 9, 1, 2, 0, -1]
```

```
def minimax(depth, index, maximizing, alpha, beta):
```

```
    if depth == 3:
```

```
        return tree[index]
```

```
    if maximizing:
```

```
        best = -math.inf
```

```
        for i in range(2):
```

```
            value = minimax(depth + 1, index * 2 + i, False, alpha, beta)
```

```
            best = max(best, value)
```

```
            alpha = max(alpha, best)
```

```
        if beta <= alpha:
```

```
            break
```

```
        return best
```

```
    else:
```

```
        best = math.inf
```

```
        for i in range(2):
```

```
            value = minimax(depth + 1, index * 2 + i, True, alpha, beta)
```

```
            best = min(best, value)
```

```
        beta = min(beta, best)
    if beta <= alpha:
        break
    return best
result = minimax(0, 0, True, -math.inf, math.inf)
print("Best Move Value:", result)
```

OUTPUT:

```
===== RESTART: C:/WASTE/SPI.py =====
Best Move Value: 5
```